Laravel Eloquent, DB Transaction & Dynamic Query Management

Panduan Teknis Best Practices LKS Web Technologies | Framework Laravel 11.x

1. Mekanisme Kerja & Best Practices Eloquent

Eloquent ORM di Laravel menyediakan pustaka pencarian dan penyaringan data yang dinamis. Dalam modul lomba LKS Web Technologies, efisiensi pencarian dan pengelolaan pagination sangat menentukan performa backend API.

A. Dynamic Query Chaining (Searching & Filtering)

Gunakan helper `when()` untuk menyusun query kondisional secara dinamis tanpa percabangan `if-else` yang rumit.

```php
public function index(Request $request)
{
    $products = Product::query()
        ->when($request->search, function ($q, $search) {
            return $q->where('name', 'like', "%{$search}%")
                     ->orWhere('description', 'like', "%{$search}%");
        })
        ->when($request->category_id, fn($q, $catId) => $q->where('category_id', $catId))
        ->when($request->min_price, fn($q, $min) => $q->where('price', '>=', $min))
        ->when($request->max_price, fn($q, $max) => $q->where('price', '<=', $max))
        ->latest()
        ->paginate($request->get('per_page', 10));

    return response()->json($products);
}
```

B. Advanced Local Scopes pada Model

Untuk menjaga controller tetap bersih (*skinny controller*), pindahkan logika pencarian dan penyaringan yang kompleks ke dalam Local Scope pada Model Eloquent.

```php
namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Builder;

class Product extends Model
{
    public function scopeFilter(Builder $query, array $filters): Builder
    {
        return $query->when($filters['search'] ?? null, function ($q, $search) {
            $q->where(function ($sub) use ($search) {
                $sub->where('name', 'like', "%{$search}%")
                    ->orWhere('brand', 'like', "%{$search}%");
            });
        })->when($filters['status'] ?? null, fn($q, $status) =>
$q->where('status', $status));
    }
}

public function search(Request $request)
{
    $products = Product::filter($request->all())->paginate(15);
    return response()->json($products);
}
```

C. Invokable Controller untuk Endpoint Khusus

Untuk controller yang hanya menangani satu tindakan tunggal (Single Action Controller), gunakan magic method `__invoke()`. Hal ini menyederhanakan deklarasi routing tanpa perlu menyebutkan nama method secara eksplisit.

```php
namespace App\Http\Controllers;

use App\Models\Product;
use Illuminate\Http\Request;

class BestSellerProductController extends Controller
{
    public function __invoke(Request $request)
    {
        $products = Product::where('is_bestseller', true)
            ->latest()
            ->take(10)
            ->get();
```

```
        return response()->json($products);
    }
}

use App\Http\Controllers\BestSellerProductController;
use Illuminate\Support\Facades\Route;

Route::get('/products/bestsellers', BestSellerProductController::class);
```

D. Implicit Route Model Binding & Custom Binding (ID vs Slug)

Laravel secara default mencari rekaman database berdasarkan primary key (`id`). Untuk pencarian berbasis slug, Kalian dapat mendefinisikan kunci kustom langsung pada definisi rute atau meng-override method `getRouteKeyName()` di dalam model.

```
PHP
use App\Models\Product;
use Illuminate\Support\Facades\Route;

Route::get('/products/{product:slug}', function (Product $product) {
    return response()->json($product);
});

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Product extends Model
{
    public function getRouteKeyName(): string
    {
        return 'slug';
    }
}

Route::get('/products/{product}', function (Product $product) {
    return response()->json($product);
});
```

E. Penanganan & Generasi Slug di Laravel

Pembuatan slug URL yang bersih dapat dilakukan secara dinamis menggunakan helper `Str::slug()`, atau secara otomatis saat model disimpan melalui method `booted()` maupun Model Observer.

```php
namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Support\Str;

class Product extends Model
{
    protected $fillable = ['name', 'slug', 'price'];

    protected static function booted(): void
    {
        static::creating(function ($product) {
            if (empty($product->slug)) {
                $product->slug = Str::slug($product->name);
            }
        });
    }
}

$slug = Str::slug('Laravel 11 REST API Guide');
```

2. Dynamic Transaction & Relationship System

Operasi database multi-tabel (seperti transaksi checkout, pendaftaran installment, dan status log) WAJIB menggunakan DB Transaction agar terhindar dari ketidakstabilan data saat terjadi kegagalan sistem.

A. DB Transaction Wrapper (Atomic Operations)

```php
use Illuminate\Support\Facades\DB;
use App\Models\InstallmentApplySociety;
use App\Models\InstallmentApplyStatus;

public function applyInstalment(Request $request)
{
    return DB::transaction(function () use ($request) {
        $society = $request->user();

        $application = InstallmentApplySociety::create([
            'society_id'         => $society->id,
            'installment_id'     => $request->installment_id,
```

```
            'available_month_id' => $request->available_month_id,
            'notes'              => $request->notes,
            'date'               => now()->toDateString(),
        ]);

        InstallmentApplyStatus::create([
            'society_id'                    => $society->id,
            'installment_id'                => $request->installment_id,
            'available_month_id'            => $request->available_month_id,
            'installment_apply_societies_id' => $application->id,
            'date'                          => now()->toDateString(),
            'status'                        => 'pending',
        ]);

        return response()->json([
            'message' => 'Applying for Instalment successful',
            'data'    => $application
        ], 201);
    });
}
```

B. Relationship Query Builder vs Dynamic Property

PHP

```
$society = Society::find(1);

$activeApplications = $society->applications()
    ->where('date', '>=', '2026-01-01')
    ->orderBy('id', 'desc')
    ->get();

$allApplications = $society->applications; // Returns Collection

foreach ($allApplications as $app) {
    echo $app->notes;
}
```

C. Konsep Atomic Operations & Mekanisme Rollback

Mekanisme `DB::transaction` menjamin prinsip atomisitas (bagian dari standar ACID) dengan pendekatan *all-or-nothing*. Seluruh operasi tulis database di dalam penutupan (*closure*) akan otomatis di-commit jika tidak terjadi kesalahan, atau secara instan dibatalkan (*rollback*) ketika sistem melempar `Exception` atau `Throwable`, menjaga integritas data dari mutasi parsial.

```php
use Illuminate\Support\Facades\DB;
use Exception;

public function processPayment(Order $order, float $amount)
{
    return DB::transaction(function () use ($order, $amount) {
        $order->update(['status' => 'paid']);

        $payment = Payment::create([
            'order_id' => $order->id,
            'amount'   => $amount,
            'paid_at'  => now(),
        ]);

        if ($amount < $order->total_price) {
            throw new Exception('Insufficient payment balance.');
        }

        return response()->json($payment, 201);
    });
}
```

D. Manajemen Race Condition & Concurrency Control

Ketika beberapa permintaan HTTP bersamaan mengakses sumber daya yang sama, *race condition* dapat memicu masalah serius seperti *double booking* atau stok minus. Laravel menyediakan `lockForUpdate()` untuk mengunci baris data di level database (Pessimistic Locking) sampai transaksi selesai diproses.

Dalam modul lomba LKS Web Technologies, Pessimistic Locking direkomendasikan untuk alur transaksi kuota kritis (seperti reservasi slot, pembelian tiket, atau pengurangan inventaris) karena menjamin konsistensi mutlak tanpa memerlukan mekanisme retry di sisi klien layaknya Optimistic Locking.

```php
use Illuminate\Support\Facades\DB;
use App\Models\EventSlot;
use App\Models\Booking;
use Exception;

public function reserveSlot(Request $request)
{
    return DB::transaction(function () use ($request) {
        $slot = EventSlot::where('id', $request->slot_id)
```

```
            ->lockForUpdate()
            ->firstOrFail();

        if ($slot->quota <= $slot->booked_count) {
            throw new Exception('Slot already fully booked.');
        }

        $slot->increment('booked_count');

        $booking = Booking::create([
            'user_id'       => $request->user()->id,
            'event_slot_id' => $slot->id,
            'booking_code'  => strtoupper(uniqid('BK-')),
        ]);

        return response()->json($booking, 201);
    });
}
```

E. Penjelasan Eloquent Relationships (One-to-One, One-to-Many, Many-to-One, Many-to-Many)

Eloquent memudahkan Kalian dalam menghubungkan antar tabel melalui relasi bawaan. Kalian dapat mendefinisikannya sebagai method di dalam model untuk menghubungkan entitas secara langsung.

1. One-to-One (hasOne & belongsTo)
Relasi satu-ke-satu menghubungkan satu baris data dengan tepat satu baris data pada tabel lain. Kalian mendefinisikan `hasOne` pada model induk dan `belongsTo` pada model anak yang memegang `foreign_key`.

PHP
```
namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\HasOne;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

class User extends Model
{
    public function profile(): HasOne
    {
        return $this->hasOne(Profile::class);
    }
```

```
}

class Profile extends Model
{
    public function user(): BelongsTo
    {
        return $this->belongsTo(User::class);
    }
}
```

2. One-to-Many & Many-to-One (hasMany & belongsTo)
Relasi satu-ke-banyak digunakan saat satu model induk memiliki banyak entitas anak. Sebaliknya, relasi banyak-ke-satu merujuk kembali dari sisi anak ke entitas induk.

```
PHP
namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\HasMany;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

class Category extends Model
{
    public function products(): HasMany
    {
        return $this->hasMany(Product::class);
    }
}

class Product extends Model
{
    public function category(): BelongsTo
    {
        return $this->belongsTo(Category::class);
    }
}
```

3. Many-to-Many (belongsToMany dengan Tabel Pivot)
Relasi banyak-ke-banyak menghubungkan dua tabel melalui tabel perantara (pivot). Kalian mendefinisikan relasi ini pada kedua model dengan `belongsToMany`.

```php
namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsToMany;

class User extends Model
{
    public function roles(): BelongsToMany
    {
        return $this->belongsToMany(Role::class, 'role_user');
    }
}

class Role extends Model
{
    public function users(): BelongsToMany
    {
        return $this->belongsToMany(User::class, 'role_user');
    }
}
```

3. Optimasi & Performance Best Practices

A. Preventing N+1 Query Problem (Eager Loading)

```php
$cars = Installment::all();
foreach ($cars as $car) {
    echo $car->brand->brand; // Query berulang tiap iterasi
}

$cars = Installment::with(['brand', 'availableMonths', 'applications.status'])
    ->where('price', '>', 100000000)
    ->get();
```

B. Fast Response Transformation via makeHidden()

```php
public function getProfile(Request $request)
{
    $society = $request->user()->load('regional:id,province,district');
```

```
    $responseData = $society->makeHidden([
        'password',
        'login_tokens',
        'created_at',
        'updated_at'
    ])->toArray();

    return response()->json([
        'message' => 'Profile retrieved successfully',
        'data'    => $responseData
    ], 200);
}
```

4. Best Practices Centralized Exception Handling di bootstrap/app.php

Pada Laravel 11, penanganan exception terpusat dikonfigurasikan langsung melalui berkas `bootstrap/app.php` menggunakan closure `->withExceptions()`. Pendekatan ini memungkinkan Kalian menyeragamkan struktur respons JSON error untuk kebutuhan RESTful API seperti penanganan `AuthenticationException` (401), `NotFoundHttpException` (404), dan `ValidationException` (422) tanpa perlu memodifikasi controller satu per satu.

PHP
```
use Illuminate\Foundation\Application;
use Illuminate\Foundation\Configuration\Exceptions;
use Illuminate\Foundation\Configuration\Middleware;
use Illuminate\Auth\AuthenticationException;
use Symfony\Component\HttpKernel\Exception\NotFoundHttpException;
use Illuminate\Validation\ValidationException;
use Illuminate\Http\Request;

return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web: __DIR__.'/../routes/web.php',
        api: __DIR__.'/../routes/api.php',
        commands: __DIR__.'/../routes/console.php',
        health: '/up',
    )
    ->withMiddleware(function (Middleware $middleware) {
        //
    })
```

```
    ->withExceptions(function (Exceptions $exceptions) {
        $exceptions->shouldRenderJsonWhen(function (Request $request, Throwable
$e) {
            return $request->is('api/*') || $request->expectsJson();
        });

        $exceptions->render(function (AuthenticationException $e, Request
$request) {
            return response()->json([
                'message' => 'Unauthenticated.'
            ], 401);
        });

        $exceptions->render(function (NotFoundHttpException $e, Request $request)
{
            return response()->json([
                'message' => 'Resource not found.'
            ], 404);
        });

        $exceptions->render(function (ValidationException $e, Request $request) {
            return response()->json([
                'message' => 'The given data was invalid.',
                'errors'  => $e->errors()
            ], 422);
        });
    })->create();
```